

Ver el error de hoy y acordar qué construimos

Un agente que mapea el catálogo a Shopee, decidido paso a paso.

War Room · Alephee × Craftech × AWS · 1/10/2026

Ver el error de hoy y acordar qué construimos

De un producto de Alephee a una publicación de Shopee.

Caso real · SKU 88904447 · Correia dentada (ACDelco) · catálogo GM Brasil

Entra: el producto (Mercado Libre)	Sale: la publicación (Shopee)	Quién lo resuelve
Categoría: Correias Dentadas (738058)	Distribuição e Correias (102278:vendor:shopee)	La tabla reference_category
Condición del ítem = Novo	Item condition = Novo (valued 14703)	La tabla de atributos y la lista del canal
Tiempo de garantía = 6 meses	Duração da Garantia = 6 Months (valued 810)	El agente: el valor equivalente de la lista
Peso = 0.18 Kilogramos	Weight = 0.18 Kilograms	El agente, con la unidad del canal
Código universal de producto = -1	No se publica: -1 significa sin dato	El guardrail
Código OEM: no viene en el producto	missing: Código OEM, sin dato en el origen (hoy salió ABS Plastic y Shopee lo rechazó)	El guardrail; después decide una persona

Ver el error de hoy y acordar qué construimos

Hoy: dos llamadas, un merge y un presupuesto que se agota.

- Categoría: la tabla `reference_category` y, si no alcanza, el modelo elige
- Atributos: dos llamadas en paralelo (con y sin referencia) y un merge por código
- USD 350 por mes: cuando se agota, se publica sin atributos

Ver el error de hoy y acordar qué construimos

Así se pide hoy la categoría.

Prompt actual de Alephee · [inputs/prompts-actuales/index.ts](#)

[inputs/prompts-actuales/index.ts](#) · líneas 4–21

```
4 export const categoryPrompt = (product: CatalogItem, categories: any, language: string): string => `
5 You are an assistant that must identify the single best matching marketplace category
6 for a given CRM product.
7
8 Follow these steps carefully:
9
10 1□□ Step 1 – Exact Match:
11 - Compare the CRM category name with each marketplace category name.
12 - If you find an **exact or nearly identical** match (ignoring case, accents, plural/singular),
13   return that category immediately.
14
15 2□□ Step 2 – Semantic Match:
16 - Only if no exact match exists, choose the category whose meaning is most related
17   to both the CRM category name and the product name.
18
19 3□□ Rules:
20 - Never invent, merge, or rename categories.
21 - Never modify the URN. You must copy it **exactly as shown** in the list below, including the full prefix like "urn:category:..."
```

Reglas correctas (no inventar, copiar el URN exacto). Lo que se va a revisar: el JSON se pide en el texto y la lista completa de categorías viaja en cada llamada.

Ver el error de hoy y acordar qué construimos

Lo que Shopee rechazó y lo que aceptó mal.

Datos reales · WarRoom.zip del 24/09 · 30 productos

Caso	Qué salió	Qué pasó
88904447	Código OEM = ABS Plastic	Rechazado: el valor no pertenece al atributo
93221445	Inmetro Certification con varios valores	Rechazado: solo admite un valor
9 de 30	Valor -1 publicado como valor	Aceptado por Shopee
98550368	14 de 15 URN sin sufijo del canal	Aceptado por Shopee

Ver el error de hoy y acordar qué construimos

Seis bloques: cada decisión se compila en una versión.

Decidimos → construimos → medimos. Nunca se pasa al bloque siguiente con algo roto.

01 Punto de partida

02 Diseñar el agente

03 V1 · responde

04 V2 · herramientas

05 V3 · control

06 La prueba

Ver el error de hoy y acordar qué construimos

¿Qué hace y qué no hace?

DECISIÓN 1

¿Qué alcance tiene el agente del war room?

A · Un canal (Shopee) y una familia de productos: categoría + atributos

B · Todos los canales conectados (Shopee, Magalu, Tienda Nube...)

C · También título, descripción y marca

PROPUESTA · A. Un canal y una familia alcanzan para decidir el método y medirlo. Los prompts de marca, título y descripción son generación libre, no mapeo: quedan fuera. Los otros canales replican el mismo diseño con sus propias tablas.

decisiones/01-alcance.md

Ver el error de hoy y acordar qué construimos

Entra un producto. Sale una propuesta de publicación.

- Entrada: el producto de Alephee (SKU, nombre, descripción, categoría legacy, atributos)
- Salida: la categoría de Shopee más los atributos con URN, valueId y valor
- Y dos listas más: missing (obligatorios sin dato) y rejected (atributos descartados, con motivo)

Ver el error de hoy y acordar qué construimos

El contrato, en código.

core/src/catalogo/modelos.py · líneas 21–37

```
21 class Faltante(_Estricto):
22     urn: str = Field(description="URN del atributo obligatorio del canal (o 'category').")
23     reason: str
24
25
26 class Descartado(_Estricto):
27     legacyId: str = Field(description="id del atributo del producto (sin el prefijo urn:attribute:).")
28     reason: str
29
30
31 class Publicacion(_Estricto):
32     """Resultado del mapeo de un producto a la publicación del canal."""
33
34     category: str | None = Field(description="URN de la categoría del canal, copiado exacto. null si no se puede resolver.")
35     attributes: list[AtributoMapeado]
36     missing: list[Faltante] = Field(description="Obligatorios del canal (o la categoría) que no se pudieron completar.")
37     rejected: list[Descartado] = Field(description="Atributos del producto que se descartan, con el motivo.")
```

Pydantic con extra="forbid": ningún campo fuera del contrato pasa. Los nombres (valued, legacyId) son los del formato de publicación de Alephee para no traducir.

Ver el error de hoy y acordar qué construimos

¿Qué recibe y qué entrega?

DECISIÓN 2

¿Cómo se informa lo que no se pudo mapear?

A · No se informa: se publica lo que hay (hoy)

B · missing y rejected con motivo, en la misma salida

C · Un archivo de log aparte que alguien revisa

PROPUESTA · B. El faltante viaja con la publicación: quien revisa ve el producto, el atributo y el motivo juntos. Un log aparte se separa del dato y nadie lo mira. La decisión 12 define qué pasa después con esas listas.

decisiones/02-contrato.md

Tomar las decisiones que definen la V1

Diseñar antes de escribir.

Siete decisiones que definen la V1

Tomar las decisiones que definen la V1

Cuatro formas de usar un modelo.

- SINGLE PROMPT | Una llamada, una respuesta. Barato y fácil de evaluar; no consulta nada
- WORKFLOW | Pasos fijos en código; el modelo participa en uno o dos. Predecible
- AGENTE | El modelo decide qué herramienta pedir y cuándo entregar, dentro de un límite de rondas
- MULTIAGENTE | Varios agentes con roles distintos. Solo cuando uno no alcanza

Tomar las decisiones que definen la V1

Las tres versiones conviven en el mismo runner.

core/src/catalogo/correr.py · líneas 24–27

```
24 def _versiones() -> dict:
25     from . import v1, v2, v3
26
27     return {"v1": v1.MapeoV1, "v2": v2.MapeoV2, "v3": v3.MapeoV3}
```

Cada versión es un Workflow que recibe MapeoStart y devuelve MapeoDone. Se comparan sobre el mismo dataset con el mismo evaluador.

Tomar las decisiones que definen la V1

¿Single prompt o agente?

La opción A ya está en la base preparada (V1) · si gana otra, se discute qué cambia

DECISIÓN 3

¿Con qué tipo de aplicación empezamos?

A · Single prompt (V1) y agregar herramientas solo cuando un fallo lo justifique

B · Agente con herramientas desde el inicio

C · Workflow determinista sin modelo

PROPUESTA · A. La V1 muestra qué resuelve el modelo solo y qué no; cada fallo justifica la capa siguiente y queda documentado. C no alcanza: elegir el valor de lista equivalente (por ejemplo, '1' → Sim) necesita interpretación.

decisiones/03-tipo-de-aplicacion.md

Tomar las decisiones que definen la V1

Un prompt tiene cinco partes.

01 Rol: quién es

02 Tarea: qué hace y con qué entrada

03 Reglas: qué nunca hace

04 Formato: cómo entrega

05 Cuando falta dato: qué hace

Tomar las decisiones que definen la V1

El prompt de la V1.

core/src/catalogo/v1.py · líneas 22–39

```
22 INSTRUCCIONES = """\
23 Eres un especialista en catalogación de autopartes para marketplaces. Recibes un producto del \
24 catálogo de Alephee (taxonomía de Mercado Libre) y debes adaptarlo a Shopee: elegir la \
25 categoría de Shopee y mapear los atributos del producto a los atributos que esa categoría espera.
26
27 Reglas:
28 - La categoría y los URN de atributos se copian exactos de las listas que recibes. Nunca inventes un URN.
29 - Si el atributo del canal tiene una lista de valores, elige el valor equivalente de esa lista y usa su id. \
30 Si ningún valor es equivalente, no lo completes.
31 - Si el atributo del canal es de texto libre, usa valueId "0" y el valor del producto tal cual.
32 - Nunca inventes valores. Solo usas datos presentes en los atributos del producto.
33 - Los valores "-1", "N/A" o vacíos significan que no hay dato.
34 - Si un atributo obligatorio del canal no se puede completar, agrégalo a "missing" con el motivo.
35 - Si un atributo del producto no se puede usar, agrégalo a "rejected" con el motivo.
36 - Cada atributo del canal aparece una sola vez.
37
38 Entrega el resultado llamando a la herramienta entregar_publicacion."""
39
```

Rol, tarea, reglas negativas explícitas (nunca inventes) y qué hacer cuando falta (missing con motivo). Comparar con el prompt actual de la lámina 4.

Tomar las decisiones que definen la V1

Buenas prácticas que cambian el resultado.

- Lo estático primero y lo variable al final: habilita la caché de prompt (F4, F7)
- Salida estructurada por esquema, no JSON pedido en el texto
- Decir qué hacer cuando no sabe, no solo qué hacer

Tomar las decisiones que definen la V1

La salida es una herramienta con esquema.

core/src/catalogo/v1.py · líneas 41–51

```
41 def _entregar(**publicacion) -> str:
42     # El resultado se lee de la tool call; la herramienta nunca se ejecuta.
43     return "ok"
44
45
46 HERRAMIENTA_SALIDA = FunctionTool.from_defaults(
47     fn=_entregar,
48     name=NOMBRE_SALIDA,
49     description="Entrega el resultado final del mapeo del producto a la publicación del canal.",
50     fn_schema=Publicacion,
51 )
```

`fn_schema=Publicacion`: el modelo solo puede entregar algo que cumpla el contrato. La función nunca se ejecuta; el resultado se lee de la llamada.

Tomar las decisiones que definen la V1

¿Cómo garantizamos el formato?

La opción B ya está en la base preparada · si gana otra, se discute qué cambia

DECISIÓN 4

¿Cómo se garantiza que la salida cumpla el contrato?

A · Pedir JSON en el texto y parsear (hoy)

B · Herramienta con esquema Pydantic: la entrega es una llamada tipada

C · Post-procesar la respuesta con expresiones regulares

PROPUESTA · B. El formato deja de ser una regla del prompt y pasa a ser un contrato que el modelo no puede violar. C arregla síntomas: si el modelo omite un campo, la regex no lo inventa.

decisiones/04-salida-estructurada.md

Tomar las decisiones que definen la V1

Dónde corre: AgentCore o contenedor propio.

Fuentes F1–F3b en docs/warroom/fuentes.md

- AGENTCORE RESUELVE | Runtime sin servidores, identidad, memoria, gateway de herramientas y observabilidad, como servicios separados que se usan juntos o no (F2)
- DUDAS | Arranque en frío sin cifra publicada, sesiones que terminan a los 15 min de inactividad y 8 h máximo (F3b), costo por CPU y memoria de la sesión (F3), región disponible
- ALTERNATIVA | Contenedor o Lambda propios: más control y más trabajo de operación; el agente es el mismo

Tomar las decisiones que definen la V1

El contrato con AgentCore son dos rutas.

core/server.py · líneas 194–199

```
194     async def ping(_: Request) -> JSONResponse:
195         return JSONResponse({"status": "Healthy"})
196
197     async def invocations(request: Request):
198         payload = await request.json()
199         message = payload.get("message")
```

Las dos rutas que exige AgentCore (F1: puerto 8080, imagen ARM64). Hoy este servidor arma el chat del template; conectar el workflow de catálogo (mapear_producto) es un pendiente.

Tomar las decisiones que definen la V1

La infraestructura declara el Runtime.

infra/sst/runtime.ts · líneas 389–406

```
389 export const runtime = new awsnative.bedrockagentcore.Runtime("AgentRuntime", {
390   // El recurso exige `[a-zA-Z][a-zA-Z0-9_]*`: stages con guion (p.ej. `pr-123`)
391   // lo violarían, así que se sanitiza cualquier carácter fuera de ese alfabeto.
392   // El prefijo es el slug del cliente en camelCase (infra/sst/nombres.ts): ese
393   // alfabeto no acepta `-`.
394   agentRuntimeName: $interpolate`${slugCamel}${$app.stage.replace(/^[a-zA-Z0-9_]/g, "_")}`,
395   agentRuntimeArtifact: {
396     containerConfiguration: {
397       // Digest PURO (repo@sha256:...): el ref canónico de docker-build combina
398       // tag y digest (repo:latest@sha256:...) – la validación de CreateRuntime
399       // lo acepta, pero el pull real de la microVM no lo resuelve y el
400       // contenedor jamás arranca (health check timeout con logs vacíos).
401       // El digest cambia por build, así que los updates siguen disparando.
402       containerUri: image.ref.apply((r) => r.replace(/:^[^@]+@/, "@")),
403     },
404   },
405   networkConfiguration: { networkMode: "PUBLIC" },
406   roleArn: role.arn,
```

Imagen ARM64 referenciada por digest, red pública y un rol propio. Se despliega con SST; el modelo llega por variable de entorno.

Tomar las decisiones que definen la V1

¿Dónde corre?

DECISIÓN 5

¿Dónde corre el agente?

A · Hoy en local; el chat de demo en AgentCore; el batch como proceso de Alephee, a decidir con el camino a producción

B · Todo en AgentCore desde el inicio

C · Todo en la infraestructura actual de Alephee

PROPUESTA · A. Hoy se construye y se mide en local. El chat va a AgentCore porque es su caso natural. Para el batch hace falta medir sesiones, costo y arranque antes de decidir; se retoma a las 16:15.

decisiones/05-donde-corre.md

Tomar las decisiones que definen la V1

Elegir el modelo: qué pesa en este caso.

- Seguir reglas y usar herramientas sin inventar: es lo que más falla hoy
- Costo por token con caché de prompt: el presupuesto actual es de USD 350 por mes
- Latencia: no es restricción (hoy el batch tarda 13 s por producto)
- Disponible en la región y habilitado en la cuenta

Tomar las decisiones que definen la V1

Qué hay en Bedrock.

Fuente F6 · Models at a glance · verificado el 30/09/2026

Proveedor	Familias	Para este caso
Anthropic	Claude 5.x (Sonnet 5.5, Opus 5.5, Fable 5.1, Sonnet 5...), 4.x (Haiku 4.5...), 3.x	Candidato principal: reglas y tool calling. Sonnet 5 elegido el 25/09; 5.5 y Haiku 4.5 a comparar
Amazon	Nova 2 Lite, Nova Premier, Pro, Lite, Micro	Alternativa de costo a medir
OpenAI	GPT-5.x, GPT-6, GPT OSS	Continuidad con el proveedor actual, dentro de AWS
Meta · Mistral · DeepSeek · Qwen · otros	Llama 3.x y 4, Mistral Large 3, DeepSeek V3.2, Qwen3...	Abiertos; evaluar si el costo lo justifica

Tomar las decisiones que definen la V1

El único módulo que sabe de Bedrock.

core/src/catalogo/llm.py · líneas 7–21

```
7 # Converse acepta `us.` y `global.` (verificado 28/09); sin prefijo no hay throughput
8 # on-demand. `us.` es el que contempla la policy del RuntimeRole.
9 MODEL_ID_POR_DEFECTO = "us.anthropic.claude-sonnet-5"
10
11
12 def crear_llm(env=os.environ) -> BedrockConverse:
13     return BedrockConverse(
14         model=env.get("MODEL_ID", MODEL_ID_POR_DEFECTO),
15         region_name=env.get("AWS_REGION", "us-east-1"),
16         # En local se usa el perfil SSO; en el Runtime no hay perfil y boto3 toma el rol.
17         profile_name=env.get("AWS_PROFILE") or None,
18         max_tokens=16000,
19         system_prompt_caching=True,
20         tool_caching=True,
21     )
```

Inference profile us.: sin prefijo, Bedrock responde ValidationException por falta de throughput on-demand (F5). Caché de system y tools activada en el cliente.

Tomar las decisiones que definen la V1

¿Qué modelo?

Tomada el 25/09 (CLAUDE.md, decisión 6) · hoy se valida o se cambia

DECISIÓN 6

¿Con qué modelo construimos y medimos?

A · Claude Sonnet 5 en Bedrock, vía Converse

B · Un modelo más chico y barato (Claude Haiku 4.5) y medir

C · Seguir con OpenAI y solo reordenar el prompt para la caché

PROPUESTA · A para construir hoy: es la decisión del 25/09 y el ID está verificado en Converse. Claude Sonnet 5.5 también está en Bedrock (F6): compararlo, junto con Haiku 4.5 (B), es una corrida más con otra variable de entorno y queda como pendiente con dueño. C mejora el costo pero no resuelve herramientas, contrato ni control.

decisiones/06-modelo.md

Tomar las decisiones que definen la V1

Stack y harness.

- Python + LlamaIndex Workflows: pasos, eventos y herramientas ya resueltos
- BedrockConverse: tool calling y caché de prompt sin código propio
- Harness de desarrollo: propone cambios en el código; Gastón los revisa y corre los tests

Tomar las decisiones que definen la V1

Un Workflow de un paso.

core/src/catalogo/v1.py · líneas 76–83

```
76 class MapeoV1(Workflow):
77     def __init__(self, llm, **kwargs):
78         # `llm` es un BedrockConverse (catalogo/llm.py) o un doble en los tests.
79         super().__init__(**kwargs)
80         self.llm = llm
81
82     @step
83     async def mapear(self, ev: MapeoStart) -> MapeoDone:
```

El LLM se inyecta: en los tests es un doble, en producción es Bedrock. El workflow no sabe de proveedores.

Tomar las decisiones que definen la V1

¿Con qué lo construimos?

Tomada el 28/09 (CLAUDE.md, decisión 9) · hoy se valida

DECISIÓN 7

¿Qué stack usamos para el agente?

A · Python + LlamaIndex Workflows + BedrockConverse

B · Llamadas directas al SDK, sin framework (como hoy)

C · Otro framework de agentes (Strands, LangGraph...)

PROPUESTA · A. Es el stack del template que Crafttech ya opera: el workflow no conoce Bedrock y cada paso se prueba con dobles. B repite el código de loop, herramientas y caché que un framework ya trae. C es válido; se elige A por continuidad con lo que ya está probado.

decisiones/07-stack.md

Tomar las decisiones que definen la V1

¿Cuándo está bien hecho?

Esquema de Shopee y salida esperada MOCK · validar con el equipo de catálogo

- Exacto: categoría correcta, ni sobra ni falta atributo, nada fuera de dominio, sin duplicados y faltantes informados
- Además se reportan precisión y recall de atributos, valores inválidos, tokens y segundos
- Dataset: 30 productos reales con su publicación actual; el esquema de atributos de Shopee y el expected son simulados

Tomar las decisiones que definen la V1

La métrica en código.

core/src/catalogo/evaluacion.py · líneas 24–34

```
24     @property
25     def exacto(self) -> bool:
26         return (
27             self.categoria_ok
28             and self.fp == 0
29             and self.fn == 0
30             and not self.invalidos
31             and not self.duplicados
32             and not self.faltantes_no_detectados
33             and not self.faltantes_de_mas
34         )
```

Un caso es exacto solo si todo se cumple a la vez. Es determinista: no llama a ningún modelo.

Tomar las decisiones que definen la V1

¿Cuál es el número que aceptamos?

DECISIÓN 8

¿Qué criterio de éxito acordamos para la prueba de las 16:15?

A · Cero valores inválidos, cero duplicados y todo obligatorio informado en los 30; los exactos se reportan pero no son condición hasta validar el expected con catálogo

B · Exactos iguales o mayores al 80 %

C · Solo precisión y recall de atributos

PROPUESTA · A. Con expected mock, 'exacto' castiga aciertos que hoy nadie mapea: el 28/09 el proceso actual dio 7 y las versiones 5, 4 y 4. Los inválidos, duplicados y obligatorios sin informar sí son errores seguros y hoy hay 47, 5 y 2. El número final se fija en la sala.

decisiones/08-criterio-de-exito.md

Tomar las decisiones que definen la V1

¿Con qué dataset?

DECISIÓN 9

¿Sobre qué datos construimos y medimos?

A · Los 30 reales del zip, con esquema y expected mock rotulados

B · Solo los 10 casos mock de borde

C · Esperar el esquema oficial de Shopee para empezar

PROPUESTA · A. Son productos reales con publicaciones reales y rechazos reales de Shopee. Lo simulado queda rotulado y se valida con catálogo después. Los 10 mock quedan como tests de borde.

decisiones/09-dataset.md

Construir, correr y leer la primera versión

V1 · el agente responde.

Solo instrucciones. Se espera que falle, y esos fallos justifican lo que sigue.

Construir, correr y leer la primera versión

Una llamada, una entrega.

core/src/catalogo/v1.py · líneas 83–100

```
83     async def mapear(self, ev: MapeoStart) -> MapeoDone:
84         respuesta = await self.llm.achat_with_tools(
85             tools=[HERRAMIENTA_SALIDA],
86             user_msg=mensaje_producto(ev.producto),
87             chat_history=[ChatMessage(role="system", content=INSTRUCCIONES)],
88             tool_required=True,
89         )
90         uso = uso_de(respuesta)
91         llamadas = [
92             ll
93             for ll in self.llm.get_tool_calls_from_response(respuesta, error_on_no_tool_call=False)
94             if ll.tool_name == NOMBRE_SALIDA
95         ]
96         if not llamadas:
97             return MapeoDone(publicacion=None, uso=uso, error=f"El modelo no llamó a {NOMBRE_SALIDA}")
98         try:
99             publicacion = Publicacion.model_validate(llamadas[0].tool_kwargs)
100         except ValidationError as exc:
```

`tool_required=True`: el modelo tiene que entregar por la herramienta. Si no la llama o la entrega no cumple el contrato, se informa el error; nunca se publica a medias.

Construir, correr y leer la primera versión

Demo · un caso real por V1.

```
scripts/correr.sh --version v1 --datos real --caso error-88904447
```

- La categoría elegida frente a la de `reference_category`
- Cada atributo: ¿de dónde salió el valor? ¿Está en la lista del canal?
- `missing` y `rejected`: ¿informa lo que no pudo o completó igual?
- Tokens de entrada: todo el contexto del canal viaja en cada llamada

Respaldo · resultados/v1-real-20260928-173758.json (corrida del 28/09, anterior a las correcciones)

Construir, correr y leer la primera versión

Qué falló en V1 y qué capa lo resuelve.

Corrida del 28/09 · 30 reales (fila 1: caso MOCK 09) · anterior a las correcciones · expected MOCK

Fallo	Dónde se vio	Capa que lo resuelve
Categoría inventada cuando no hay referencia	Caso mock 09 (sin categoría de origen)	V2 · la tabla por herramienta
Valor fuera de la lista del canal	19 valores en los 30 reales	V3 · guardrail
Obligatorio sin informar en missing	2 casos en los 30 reales	V3 · guardrail
~22.000 tokens de entrada por producto	Todos los casos	V2 · herramientas + caché de prompt

Decidir qué resuelve la tabla y conectarla

V2 · herramientas.

El agente consulta antes de decidir.

Decidir qué resuelve la tabla y conectarla

Una herramienta es una función que el modelo pide y el código ejecuta.

01 El modelo pide `buscar_categoria('urn:category:1106872')`

02 El programa ejecuta la función: consulta la tabla

03 Devuelve `{encontrada: true, urn, name}` o `{encontrada: false, motivo}`

04 El modelo sigue con ese dato, no con su memoria

Decidir qué resuelve la tabla y conectarla

La fuente se abstrae; las herramientas no cambian.

core/src/catalogo/herramientas.py · líneas 17–34

```
17 class FuenteCatalogo(Protocol):
18     def categoria_destino(self, id_legacy: str) -> dict | None: ...
19
20     def destinos_atributo(self, id_legacy: str) -> list[dict]: ...
21
22     def esquema(self, categoria_urn: str) -> dict | None: ...
23
24
25 class FuenteArchivos:
26     """Lee data/mock o data/real (según DATA_DIR). Hace de API interna de Alephee."""
27
28     def categoria_destino(self, id_legacy: str) -> dict | None:
29         return datos.cargar_referencia_categorias().get(datos.id_categoria(id_legacy))
30
31     def destinos_atributo(self, id_legacy: str) -> list[dict]:
32         return datos.cargar_referencia_atributos().get(datos.id_legacy(id_legacy), [])
33
34     def esquema(self, categoria_urn: str) -> dict | None:
```

Hoy FuenteArchivos lee data/real; mañana una clase que llame a la base o a un endpoint interno de Alephee. Las herramientas y el agente no se tocan.

Decidir qué resuelve la tabla y conectarla

Una herramienta bien descrita.

core/src/catalogo/herramientas.py · líneas 42–48

```
42 def crear_herramientas(fuente: FuenteCatalogo) -> list[FunctionTool]:
43     def buscar_categoria(categoria_legacy: str) -> str:
44         """Devuelve la categoría de Shopee que la tabla reference_category asigna a la categoría
45         legacy del producto (por ejemplo 'urn:category:734701'). Si la tabla no la tiene, lo dice."""
46         fila = fuente.categoria_destino(categoria_legacy)
47         if fila is None:
48             return _json({"encontrada": False, "motivo": "la categoría legacy no está en reference_category"})
```

El docstring es lo que lee el modelo: qué devuelve, con qué formato de entrada y qué pasa si no encuentra. La respuesta negativa es explícita, nunca vacía.

Decidir qué resuelve la tabla y conectarla

¿Qué decide la tabla y qué decide el agente?

En la base preparada: la categoría se fija en código (V3); los campos de la tabla los aplica el modelo por instrucción

DECISIÓN 10

¿Qué decide la tabla y qué decide el agente?

- A · La tabla fija la categoría y los campos; el agente solo elige el valor equivalente de la lista y cubre lo que la tabla no tiene
- B · El agente puede corregir la tabla si cree que está mal
- C · Todo por tabla; sin modelo

PROPUESTA · A. La tabla manda (acuerdo del 25/08 con Juan David). Hoy la categoría se impone en código (V3) y la tabla de atributos la consulta el agente por herramienta, desambiguada por categoría en código; imponer también los campos por código es el siguiente paso. Lo que la tabla no cubre (valores de lista, atributos sin referencia) es lo único que decide el agente.

decisiones/10-tabla-vs-agente.md

Decidir qué resuelve la tabla y conectarla

El loop tiene un límite y una salida garantizada.

core/src/catalogo/v2.py · líneas 82–96

```
82     for ronda in range(MAX Rondas):
83         # En la última ronda solo queda la herramienta de entrega: nunca termina sin respuesta.
84         ultima = ronda == MAX Rondas - 1
85         respuesta = await self.llm.achat_with_tools(
86             tools=[HERRAMIENTA_SALIDA] if ultima else [*self.herramientas, HERRAMIENTA_SALIDA],
87             user_msg=None,
88             chat_history=historial,
89             tool_required=True,
90             allow_parallel_tool_calls=not ultima,
91         )
92         for clave, valor in uso_de(respuesta).items():
93             uso[clave] = uso.get(clave, 0) + valor
94         llamadas = self.llm.get_tool_calls_from_response(respuesta, error_on_no_tool_call=False)
95         usadas += [ll.tool_name for ll in llamadas]
96         if not llamadas:
```

Seis rondas como máximo. En la última solo queda la herramienta de entrega y se desactivan las llamadas en paralelo. Si aun así no entrega, el error es explícito.

Decidir qué resuelve la tabla y conectarla

Caché de prompt: lo estático se paga una vez.

core/src/catalogo/v2.py · líneas 70–80

```
70     async def mapear(self, ev: MapeoStart) -> MapeoDone:
71         por_nombre = {t.metadata.name: t for t in self.herramientas}
72         # El punto de caché después del producto hace que cada ronda del loop reuse
73         # instrucciones + herramientas + producto en vez de volver a pagarlos.
74         historial = [
75             ChatMessage(role="system", content=self.instrucciones),
76             ChatMessage(role="user", blocks=[TextBlock(text=mensaje_producto(ev.producto)),
77                                             CachePoint(cache_control=CacheControl(type="default"))]),
78         ]
79         uso: dict = {}
80         usadas: list[str] = []
```

Instrucciones, herramientas y producto quedan antes del punto de caché: las seis rondas del loop reutilizan ese prefijo en vez de volver a pagarlo (F4).

Decidir qué resuelve la tabla y conectarla

Costo por producto, medido.

Corrida del 28/09 · 30 reales · anterior a las correcciones · tokens de entrada por producto

Versión	Entrada sin caché	Leída de caché	Segundos
V1	~22.000	—	12,3
V2	~1.600	~12.000	18,3

Decidir qué resuelve la tabla y conectarla

¿Cuánto puede costar?

Abierta: hoy no hay tope ni medición completa en la base preparada

DECISIÓN 11

¿Cómo tratamos la restricción de costo del modelo?

A · Tope mensual acordado con Alephee y medición por producto: tokens de entrada, salida, caché leída y escrita

B · Sin tope: se mide después

C · El tope de hoy (USD 350) y publicar sin atributos al agotarse

PROPUESTA · A. Hoy el tope corta la calidad (C). Con caché de prompt y caché por SKU (decisión 13) el costo por producto baja; el número se fija con una corrida completa medida, no con esta tabla.

decisiones/11-costo.md

Decidir qué resuelve la tabla y conectarla

Demo · el mismo caso por V2.

```
scripts/correr.sh --version v2 --datos real --caso error-88904447
```

- Qué herramientas pidió y en qué orden (herramientas_usadas)
- La categoría ahora sale de la tabla, no del modelo
- Código OEM: ¿sigue siendo ABS Plastic?
- Tokens leídos de caché frente a los nuevos

Respaldo · resultados/v2-real-20260928-181509.json (corrida del 28/09, anterior a las correcciones)

Decidir qué pasa cuando el agente no sabe

V3 · control.

Qué pasa cuando el agente no sabe.

Decidir qué pasa cuando el agente no sabe

Guardrail: una comprobación en código, no otra instrucción.

- REVISAR | Lista los problemas en lenguaje claro y se los devuelve al agente, que tiene una ronda para corregir
- LIMPIAR | Red final: descarta lo inválido con motivo y marca los obligatorios que faltan
- NUNCA | Publica un valor fuera de la lista del canal, un -1 ni un duplicado. No verifica que el valor sea verdad: eso sigue siendo del esquema y de catálogo

Decidir qué pasa cuando el agente no sabe

Qué mira el validador.

core/src/catalogo/guardrails.py · líneas 11–23

```
11 def _problema(attr: dict, esquema: dict[str, dict]) -> str | None:
12     definicion = esquema.get(attr["urn"])
13     if definicion is None:
14         return f"{attr['urn']} no es un atributo de esta categoría"
15     if str(attr.get("value", "")).strip() in SIN_DATO:
16         return f"{attr['urn']} tiene un valor sin dato ('{attr.get('value')}'); no se publica"
17     dominio = {v["id"]: v["name"] for v in definicion.get("values", [])}
18     if dominio and str(attr.get("valueId")) not in dominio:
19         return f"{attr['urn']}: el valor '{attr.get('value')}' no está en la lista del canal"
20     if dominio and attr.get("value") != dominio[str(attr.get("valueId"))]:
21         return f"{attr['urn']}: el nombre del valor no coincide con su ID en el canal"
22     return None
23
```

Cuatro comprobaciones por atributo: que exista en la categoría, que tenga dato, que el valueId esté en la lista del canal y que el nombre coincida con ese ID.

Decidir qué pasa cuando el agente no sabe

La red final no inventa: descarta y marca.

core/src/catalogo/guardrails.py · líneas 41–57

```
41 def limpiar(publicacion: dict, esquema: dict[str, dict]) -> dict:
42     atributos, descartados, vistos = [], list(publicacion["rejected"]), set()
43     for a in publicacion["attributes"]:
44         motivo = _problema(a, esquema) or (f"{a['urn']} duplicado" if a["urn"] in vistos else None)
45         if motivo:
46             descartados.append({"legacyId": a["urn"], "reason": f"guardrail: {motivo}"})
47             continue
48         vistos.add(a["urn"])
49         atributos.append(a)
50     faltantes = [m for m in publicacion["missing"]
51                  if m["urn"] not in vistos and not (m["urn"] == "category" and publicacion["category"])]
52     ya = {m["urn"] for m in faltantes}
53     faltantes += [{"urn": u, "reason": "obligatorio sin dato válido (guardrail)"}
54                  for u, d in esquema.items() if d.get("mandatory") and u not in vistos and u not in ya]
55     if not publicacion["category"] and "category" not in ya:
56         faltantes.append({"urn": "category", "reason": "sin categoría de referencia; requiere revisión"})
57     return {**publicacion, "attributes": atributos, "missing": faltantes, "rejected": descartados}
```

Lo que no pasa va a rejected con el motivo del guardrail; lo obligatorio sin dato válido va a missing; sin categoría de referencia, se pide revisión.

Decidir qué pasa cuando el agente no sabe

¿Qué hace cuando no sabe?

La opción B ya está en la base preparada (V3) · C se construye sobre B

DECISIÓN 12

¿Qué hace el agente cuando no puede completar un atributo obligatorio?

A · Publicar sin el atributo (hoy)

B · Faltante explícito con motivo: la publicación sale con missing y alguien decide

C · Bloquear la publicación hasta revisión humana

PROPUESTA · B. Reemplaza el publicar sin atributos. Quién recibe missing y si bloquea o no la publicación es la integración con la plataforma de Alephee: pendiente con dueño. C es una política válida que se puede construir sobre B.

decisiones/12-cuando-no-sabe.md

Decidir qué pasa cuando el agente no sabe

Memoria: correcciones del equipo de catálogo.

- Una corrección dice: en esta categoría, este valor del producto va a este atributo con este valor del canal
- El agente la consulta como una herramienta más y manda sobre su criterio
- Hoy es un archivo JSON local; en producción, AgentCore Memory o una tabla de Alephee

Decidir qué pasa cuando el agente no sabe

Una corrección vacía la caché.

core/src/catalogo/memoria.py · líneas 31–41

```
31     def corregir(self, categoria: str, urn: str, valor_producto: str, value_id: str, value: str,
32                 autor: str = "catálogo") -> dict:
33         """Guarda que, en esa categoría, el valor del producto `valor_producto` va a `urn` = `value`."""
34         correccion = {"categoria": categoria, "urn": urn, "valorProducto": valor_producto,
35                      "valueId": value_id, "value": value, "autor": autor, "fecha": date.today().isoformat()}
36         todas = [c for c in self._leer(self.correcciones_archivo)
37                  if not (c["categoria"] == categoria and c["urn"] == urn and c["valorProducto"] == valor_producto)]
38         self._escribir(self.correcciones_archivo, [*todas, correccion])
39         # Una corrección puede cambiar cualquier mapeo guardado: la caché se vacía.
40         self._escribir(self.cache_archivo, {})
41         return correccion
```

La corrección se guarda con autor y fecha y reemplaza a la anterior del mismo atributo. Como puede cambiar cualquier mapeo guardado, la caché de mapeos se vacía.

Decidir qué pasa cuando el agente no sabe

Mismo SKU, misma salida.

core/src/catalogo/v3.py · líneas 62–79

```

62     async def mapear(self, ev: MapeoStart) -> MapeoDone:
63         cats = ev.producto.get("categories") or []
64         clave = (ev.producto.get("sku"), id_categoria(cats[0]["urn"])) if cats else ""
65         referencia = self.fuente.categoria_destino(clave[1]) if clave[1] else None
66         self.categoria_referencia = referencia["urn"] if referencia else None
67         self.reintentos = 1
68         if not self.categoria_referencia or self.fuente.esquema(self.categoria_referencia) is None:
69             return MapeoDone(publicacion={
70                 "category": self.categoria_referencia, "attributes": [], "rejected": [],
71                 "missing": [{"urn": "category", "reason": "No hay referencia o esquema de categoría disponible; requiere revisión"}],
72                 }, herramientas_usadas=["referencia_categoria"])
73         if self.usar_cache and (guardado := self.memoria.en_cache(*clave)):
74             if guardado["category"] == self.categoria_referencia and not guardrails.revisar(guardado, self._esquema(guardado)):
75
76                 return MapeoDone(publicacion=guardrails.limpiar(guardado, self._esquema(guardado)), herramientas_usadas=["cache"])
77         done = await MapeoV2.mapear(self, ev)
78         if self.usar_cache and done.publicacion is not None:
79             self.memoria.guardar_cache(*clave, done.publicacion)
80         return done

```

La tabla fija la categoría antes de llamar al modelo; sin referencia o sin esquema, se devuelve el faltante sin invocarlo. Si hay caché válida para SKU + categoría legacy, el modelo tampoco se llama.

Decidir qué pasa cuando el agente no sabe

¿Cómo garantizamos determinismo?

La opción A ya está en la base preparada (clave SKU + categoría legacy) · falta la invalidación por tablas

DECISIÓN 13

¿Cómo garantizamos que el mismo SKU dé la misma salida?

A · Caché por SKU + canal; se invalida al corregir o al cambiar las tablas

B · Seed fijo (lo que se intentó hoy; los parámetros exactos siguen sin confirmar)

C · Recalcular siempre y aceptar variación

PROPUESTA · A. Un mapeo por SKU y canal. B reduce la variación pero no la elimina y sigue pagando cada corrida. La clave actual es SKU + categoría legacy; falta sumar versión de las tablas y aislamiento por cuenta antes de producción.

decisiones/13-determinismo-y-cache.md

Decidir qué pasa cuando el agente no sabe

Demo · corregir y repetir.

```
scripts/corregir.sh --categoria <urn-categoria> --urn <urn-atributo> --valor-producto <valor>  
--value-id <id> --value <nombre> scripts/correr.sh --version v3 --datos real --caso error-88904447  
--cache
```

- buscar_correcciones aparece en herramientas_usadas
- El valor corregido sale tal cual lo cargó catálogo
- rejected explica cada descarte del guardrail
- Segunda corrida del mismo caso: herramientas_usadas = [cache] y cero tokens

Respaldo · resultados/v3-real-20260928-180524.json (corrida del 28/09, anterior a las correcciones)

Medir contra el criterio y repartir lo que sigue

La prueba.

Hoy contra V1, V2 y V3, sobre los 30, con el criterio de la decisión 8.

Medir contra el criterio y repartir lo que sigue

Resultados (se completan en vivo).

Columna Hoy: publicaciones exportadas, expected MOCK. Las demás se llenan con la corrida del día

Métrica	Hoy	V1	V2	V3
Casos exactos	7	.	.	.
Categoría correcta	28	.	.	.
Valores inválidos (-1 o fuera de lista)	47	.	.	.
Duplicados	5	.	.	.
Obligatorios sin informar	2	.	.	.

Medir contra el criterio y repartir lo que sigue

Cómo leer la tabla.

- Exactos bajos en todas las columnas: el expected hereda las omisiones del proceso actual y castiga aciertos que hoy nadie mapea
- Inválidos y duplicados sí son errores seguros: la V3 no entrega ninguno detectable por este evaluador (28/09)
- Categoría: la V3 dio 28/30 en la corrida vieja; la corrección del 28/09 la fija desde la tabla y hay que volver a medir

Medir contra el criterio y repartir lo que sigue

Camino a producción.

01 Esquema oficial de Shopee y expected validado con el equipo de catálogo

02 Integración: la API pública lee el producto (F8); escribir publicaciones y leer tablas necesita acceso interno

03 Decidir dónde corre el batch (decisión 5) y la política de missing (decisión 12)

04 Medir el costo completo y fijar el tope (decisión 11); probar Haiku 4.5 (decisión 6)

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (1 a 7).

N	Decisión	Archivo
1	Alcance: un canal, una familia, categoría + atributos	decisiones/01-alcance.md
2	Contrato: missing y rejected con motivo	decisiones/02-contrato.md
3	Tipo de aplicación: single prompt y escalar	decisiones/03-tipo-de-aplicacion.md
4	Salida estructurada por herramienta + Pydantic	decisiones/04-salida-estructurada.md
5	Dónde corre: local hoy, chat en AgentCore, batch a medir	decisiones/05-donde-corre.md
6	Modelo: Claude Sonnet 5; Haiku 4.5 a probar	decisiones/06-modelo.md
7	Stack: Python + LlamaIndex Workflows + BedrockConverse	decisiones/07-stack.md

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (8 a 13).

N	Decisión	Archivo
8	Criterio de éxito: el número de la pizarra	decisiones/08-criterio-de-exito.md
9	Dataset: 30 reales con mock rotulado	decisiones/09-dataset.md
10	La tabla manda; el agente elige valores y lo no cubierto	decisiones/10-tabla-vs-agente.md
11	Costo: tope acordado y medición por producto	decisiones/11-costo.md
12	Cuando no sabe: faltante explícito, nunca inventar	decisiones/12-cuando-no-sabe.md
13	Determinismo: caché por SKU + canal con invalidación	decisiones/13-determinismo-y-cache.md

Medir contra el criterio y repartir lo que sigue

Quién hace qué, para cuándo.

Se completa en la sala.